Skill First Labs

Tasks for Shortlisting

Milestone 1: CSS Design-Token Extractor | Milestone 2: Evidence-First CSS Agent

Submitted by: **Sai Ramana D** | B.Tech AI & ML, Saveetha School of Engineering

Deadline: 24 Oct 2026, 08:00 PM

At a glance	
Language / deps	Python 3.9+, standard library only (runs offline, nothing to install)
Milestone 1	A hand-written CSS scanner and analyser that outputs structured design knowledge: resolved variables, colours by real usage, typography, breakpoints, dead overrides, contrast failures, palette drift.
Milestone 2	An agent with 8 tools built on Milestone 1. It answers only from tool evidence, abstains when data is missing, validates its own answers with a GroundingGuard and proves repeatability with fingerprints.
Verification	25 unit tests, all passing; 11-question scripted demo, every answer identical across 3 from-scratch runs.

Milestone 1 - CSS Parser

The idea. A parser that only dumps text is not useful. I treat a stylesheet as a small database of design decisions and extract the answers a developer actually wants: *what is our palette, is it consistent, is anything dead, is it accessible?*

What is extracted

Output	Why it is meaningful
Variables, **resolved**	Every `--token` with its full `var()` chain expanded (fallbacks honoured). Circular and undefined references are flagged, not crashed on.
Colours by **real usage**	rgb/hsl/hex/named colours normalised to hex. A `color: var(--brand)` counts towards #1a73e8, with the variable names that led there.
Typography	Font families (first in stack) and sizes, with a px estimate for rem/em.
Breakpoints	@media widths converted to px (min/max-width and range syntax) and how many rules live in each.
Dead overrides	Declarations made useless by a later one, honouring !important.
Contrast	WCAG ratio of text vs background in the same rule; flags AA failures.
Palette drift	Near-duplicate colours (e.g. #1a73e8 vs #1b74e9), a common design-system smell.
Specificity	Most specific selectors, i.e. the ones that will be hardest to override.

Required explanation (3-4 lines)

What I extract and why: design tokens rather than raw text: resolved variables, colours counted by real usage, fonts, breakpoints, dead overrides, contrast and near-duplicate colours. These answer real questions about a stylesheet, and resolving `var()` first keeps the counts honest.
Edge case I thought about: `}`, `;` and `/*` can appear inside strings and `url(data:...;base64,...)`, which breaks the usual split-on-`;/}` approach. My scanner is string-, paren- and comment-aware (and `fill: url(#abc)` is not mistaken for the colour #abc). Circular variables and unclosed blocks give warnings instead of crashes.

How it works

1) **strip_comments** blanks comments but keeps newlines, so line numbers stay exact. 2) A scanner (`_scan`, `_match_brace`) walks the text tracking string and parenthesis state, so delimiters inside them are ignored. 3) A recursive rule parser handles nested @media/@supports, statement at-rules (@import) and @font-face, recording a line number for every rule and declaration. 4) **Analysis** resolves variables in the right scope (root variables, overlaid with variables declared in the same rule; conditional :root overrides inside @media do not replace unconditional ones) and builds the report.

Sample input (sample/site.css, abridged)

```
:root {
  --brand: #1a73e8;
  --brand-hover: #1b74e9;          /* near-duplicate of --brand (palette drift) */
  --text: #202124;
  --muted: #9aa0a6;
  --surface: hsl(0, 0%, 98%);
  --space-2: 0.5rem;
  --font-body: "Inter", system-ui, sans-serif;
  --accent: var(--accent-alt);     /* circular on purpose */
  --accent-alt: var(--accent);
  --shadow: 0 2px 6px rgba(0, 0, 0, .25);
}

/* tricky: braces, semicolons and a comment marker inside strings / url() */
body {
  font-family: var(--font-body);
  font-size: 1rem;
  color: var(--text);
  background: var(--surface);
}
.quote::before { content: "} not a rule end; /* not a comment */"; }
.logo { background: url() no-repeat; fill: url(#abc); }

.btn { color: white; background-color: var(--brand); padding: var(--space-2) 1rem; font-size: 14px; }
.btn { background-color: var(--brand-hover); }          /* overrides line above */
.btn:hover { background-color: #0b57d0; }
.btn.muted { color: var(--muted); background-color: white !important; }   /* low contrast */
nav#main .item.active > a:not(.disabled) { color: crimson; border: 1px solid rgba(26, 115, 232, 0.4); }
h1 { font-family: Georgia, serif; font-size: 2rem; color: var(--missing, #333); }
.card { box-shadow: var(--shadow); color: var(--undefined-token); }
```

Milestone 1 output on the sample file

```
== CSS Design-Token Report: sample/site.css ==
15 rules | 37 declarations | 12 variables | 1 !important | 3 warnings

-- Variables (resolved) --
  --brand: #1a73e8  (line 6, :root)
  --brand-hover: #1b74e9  (line 7, :root)
  --text: #202124  (line 8, :root)
  --muted: #9aa0a6  (line 9, :root)
  --surface: hsl(0, 0%, 98%)  (line 10, :root)
  --space-2: 0.5rem  (line 11, :root)
  --font-body: "Inter", system-ui, sans-serif  (line 12, :root)
  --accent: var(--accent-alt) -> var(--accent)  (line 13, :root)   [CIRCULAR]
  --accent-alt: var(--accent) -> var(--accent-alt)  (line 14, :root)   [CIRCULAR]
  --shadow: 0 2px 6px rgba(0, 0, 0, .25)  (line 15, :root)
  --surface: #121212  (line 44, :root in @media (prefers-color-scheme: dark))
  --text: #e8eaed  (line 44, :root in @media (prefers-color-scheme: dark))

-- Colors by real usage --
  #ffffff          x2
  #000000@0.25     x1   via --shadow
  #0b57d0          x1
  #1a73e8          x1   via --brand
  #1a73e8@0.4      x1
  #1b74e9          x1   via --brand-hover
  #202124          x1   via --text
  #333333          x1
  #9aa0a6          x1   via --muted
  #dc143c          x1
  near-duplicates: #1a73e8~#1b74e9, #ffffff~#fafafa

-- Typography --
  family Georgia x1
  family Inter x1
  sizes: 0.875rem(14.0px), 14px, 1rem(16.0px), 2rem(32.0px)

-- Breakpoints --
  @media (max-width: 768px)  (2 rules)
  @media (min-width: 1200px) and (max-width: 1600px)  (1 rules)
  @media (prefers-color-scheme: dark)  (1 rules)

-- Dead / overridden declarations --
  .btn {background-color} winner=var(--brand-hover) (line 29); dead: var(--brand) (line 28)

-- Contrast (color vs background in same rule) --
  body: #202124 on #fafafa = 15.42:1 OK
  .btn: #ffffff on #1a73e8 = 4.51:1 OK
  .btn.muted: #9aa0a6 on #ffffff = 2.64:1 FAILS AA

-- Most specific selectors --
  nav#main .item.active > a:not(.disabled)  (1, 3, 2)  (line 32)
  .btn:hover  (0, 2, 0)  (line 30)
  .btn.muted  (0, 2, 0)  (line 31)

-- Warnings --
  line 13: --accent: circular reference: --accent -> --accent-alt -> --accent
  line 14: --accent-alt: circular reference: --accent-alt -> --accent -> --accent-alt
  line 34: color: undefined variable --undefined-token
```

Reading the output: **#1a73e8 (via --brand)** shows variable-aware counting; **#1a73e8~#1b74e9** is flagged as palette drift; **.btn.muted 2.64:1 FAILS AA** is a real accessibility finding; the **CIRCULAR** pair is detected and reported; and the tricky `content: "}..."` and `url(data:...;base64,...)` lines parse correctly.

Milestone 2 - AI Agent

The idea: an Evidence-First agent. It answers questions about a CSS file by calling tools on top of Milestone 1, never from a model's own knowledge. Facts come only from tool output; everything else is an abstention. The same design works with no model at all (deterministic planner) and with an LLM (Claude chooses the tools, but its answer is still verified).

Architecture

```
question --> planner --> tools --> composer --> GroundingGuard --> Answer
              |          |                          |             (text + evidence + fingerprint)
     rule-based or    8 pure functions          every hex / number /
     LLM (optional)   over Milestone 1          --variable must appear
                      status: found |           in the tool output
                      not_found | error
```

Tool	Returns
get_variable	raw + resolved value, scope, line, circular/ok status
list_colors	colours by usage, via-variables, near-duplicates
list_fonts / list_breakpoints	font families and sizes / @media queries in px
find_overrides	dead vs winning declarations (optional property filter)
get_rule	declarations of a selector with resolved values and specificity
contrast_check	WCAG ratio for two colours (hex, rgb, named or --var), or every pair in the file
file_stats	rules, declarations, variables, !important, warnings

Required explanation (3-4 lines)

It never makes up an answer: tools are the only source of facts. If no tool applies or all return `not_found`, the agent abstains ("I don't have that data") and lists what does exist. A GroundingGuard also checks that every hex colour, number and `--variable` in the final answer appears in the tool output; otherwise the answer is rejected, and an LLM answer with no tool call at all is discarded.
Same answer twice: tools are pure functions, the offline planner/composer have no randomness (LLM mode uses temperature 0), and each answer has a fingerprint of file + tool trace + text. `--check` rebuilds everything from the raw file and asks N times, comparing fingerprints; the tests do this for answerable, missing-data and out-of-scope questions.

Behaviour on the hard cases

Question	What the agent does
What is --brand?	Calls get_variable, answers #1a73e8 (line 6) with evidence.
What is --brand-dark? (not in file)	Tool returns not_found, so it **abstains** and lists the variables that do exist; it invents no value.
What is the capital of France?	No tool applies, so it **abstains** and explains its scope.
What does --accent resolve to? (circular)	Reports it is a circular reference and cannot be resolved.
Contrast between --muted and white?	contrast_check: 2.64:1, fails AA (min 4.5).
LLM mode, model invents #ff00ff	GroundingGuard rejects it and returns the evidence-only answer (unit-tested with a fake model).

Milestone 2 demo output (excerpt)

```
Q: What is --brand?
A: `--brand` = `#1a73e8` (line 6, scope :root).
   evidence: get_variable(name='--brand') -> found lines [6]
   fingerprint: 71c82da94576
   repeat x3 from scratch: IDENTICAL ['71c82da94576', '71c82da94576', '71c82da94576']

Q: What does --brand-hover resolve to and is it the same as --brand?
A: `--brand-hover` = `#1b74e9` (line 7, scope :root).
`--brand` = `#1a73e8` (line 6, scope :root).
   evidence: get_variable(name='--brand-hover') -> found lines [7]
   evidence: get_variable(name='--brand') -> found lines [6]
   fingerprint: 86bebc9bb3fc
   repeat x3 from scratch: IDENTICAL ['86bebc9bb3fc', '86bebc9bb3fc', '86bebc9bb3fc']

Q: Which colors are used the most?
A: 11 distinct colors are used. Top by usage: #ffffff (x2), #000000@0.25 (x1, via --shadow), #0b57d0 (x1), #1a73e8 ...
   evidence: list_colors() -> found lines [22, 23, 28, 29, 30, 31, 32, 33, 34]
   fingerprint: 5a7fe5071607
   repeat x3 from scratch: IDENTICAL ['5a7fe5071607', '5a7fe5071607', '5a7fe5071607']

Q: What is the contrast between --muted and white?
A: #9aa0a6 on #ffffff has a contrast ratio of 2.64:1 - fails WCAG AA for normal text (min 4.5) and fails for large ...
   evidence: contrast_check(foreground='--muted', background='white') -> found

   fingerprint: 9b395bd72d0a
   repeat x3 from scratch: IDENTICAL ['9b395bd72d0a', '9b395bd72d0a', '9b395bd72d0a']

Q: What is --brand-dark?
A: I don't have that data: --brand-dark is not defined in sample/site.css. Defined variables: --accent, --accent-al...
   [abstained - no supporting data]
   evidence: get_variable(name='--brand-dark') -> not_found
   fingerprint: fdf40fef116a
   repeat x3 from scratch: IDENTICAL ['fdf40fef116a', 'fdf40fef116a', 'fdf40fef116a']

Q: What is the capital of France?
A: I don't have data to answer that, so I won't guess. I can only answer questions about the CSS file I was given: ...
   [abstained - no supporting data]
   fingerprint: f2964c7d76b8
   repeat x3 from scratch: IDENTICAL ['f2964c7d76b8', 'f2964c7d76b8', 'f2964c7d76b8']

Q: What does --accent resolve to?
A: `--accent` = `var(--accent-alt)` (line 13, scope :root) - it is a CIRCULAR reference and cannot be resolved.
   evidence: get_variable(name='--accent') -> found lines [13]
   fingerprint: 000fe5841b50
   repeat x3 from scratch: IDENTICAL ['000fe5841b50', '000fe5841b50', '000fe5841b50']
```

Every answer prints its **evidence** (tool, arguments, lines used) and a **fingerprint**; the 'repeat x3 from scratch' line proves identical output across independent runs.

Testing and how to run

Area	What the 25 tests cover
Parser (13)	delimiters in strings/url(), comment handling and line numbers, colour normalisation (hex, rgb, hsl, alpha), url(#frag) not a colour, var chains and fallbacks, circular/undefined, breakpoints, !important overrides, conditional :root, WCAG maths, specificity, malformed input never crashes (9 broken inputs), unclosed blocks
Agent (12)	answers from tools, abstains on missing data, abstains out of scope, circular variables, named-colour contrast, compound selectors, **identical answers over repeated fresh runs**, guard rejects invented values, guard accepts faithful text, LLM answer with no tool call discarded, unknown tool / bad arguments handled

```
python milestone1.py sample/site.css
python milestone2.py sample/site.css --demo
python milestone2.py sample/site.css "What is --brand?" --check
python -m unittest discover -s tests -v          # 25 tests, all pass
# optional: ANTHROPIC_API_KEY=... python milestone2.py sample/site.css "..." --llm
```

Limitations and next steps

CSS nesting is skipped with a warning; @keyframes bodies and the `font:` shorthand are not analysed; specificity is an approximation; the built-in named-colour table is a subset; the offline planner is keyword-based, which is why the optional LLM mode exists for free-form phrasing. Natural next steps: parse nested CSS, support `@layer` cascade order, and add a JSON-schema for the report.